

Padrões de Projeto (Design Patterns)

Conceitos e Aplicações em Dart

Programação Orientada a Objetos — 2º Bimestre

Douglas Clayton da Silva

1. Introdução

Design Patterns (Padrões de Projeto) são soluções reutilizáveis para problemas recorrentes no design de software orientado a objetos. Eles não representam código pronto para copiar, mas sim modelos — receitas que descrevem como estruturar classes e objetos para resolver um tipo específico de problema de forma elegante e escalável.

O conceito foi formalizado em 1994 pelo livro "Design Patterns: Elements of Reusable Object-Oriented Software", escrito por Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides, conhecidos como Gang of Four (GoF). A obra catalogou 23 padrões divididos em três categorias.

Criacionais	Estruturais	Comportamentais
Como criar objetos (Singleton, Factory, Builder...)	Como organizar classes (Adapter, Decorator, Facade...)	Como classes se comunicam (Strategy, Observer, Command...)

O uso de padrões de projeto traz benefícios concretos: código mais fácil de manter e evoluir, vocabulário comum entre desenvolvedores, e separação clara de responsabilidades entre as classes. Em Dart, a linguagem oferece recursos como classes abstratas, interfaces (implements), mixins (with) e construtores factory que facilitam a implementação desses padrões.

2. Singleton (Criacional)

Definição: garante que uma classe possua apenas uma instância em todo o programa, oferecendo um ponto global de acesso a ela.

Quando usar: quando exatamente um objeto é necessário para coordenar ações no sistema — como configurações globais, conexão com banco de dados ou gerenciador de log.

Implementação em Dart

A estratégia em Dart combina três elementos: um atributo estático que guarda a instância, um construtor privado (impedindo criação externa) e um construtor factory que retorna sempre a mesma instância.

```
class Configuracao {  
  // Atributo estático que guarda a única instância  
  static final Configuracao _instancia = Configuracao._interno();  
  
  // Construtor privado – impede new Configuracao() fora da classe
```

```

Configuracao._interno();

// factory sempre retorna a instância existente
factory Configuracao() => _instancia;

String tema = 'escuro';
}

void main() {
  var c1 = Configuracao();
  var c2 = Configuracao();
  c1.tema = 'claro';
  print(c2.tema);           // claro – é o mesmo objeto
  print(identical(c1, c2)); // true
}

```

Ponto-chave: o construtor factory em Dart é o mecanismo natural para implementar Singleton, pois pode retornar uma instância existente em vez de criar uma nova.

3. Factory Method (Criacional)

Definição: define uma interface para criação de objetos, mas deixa as subclasses ou um método especializado decidirem qual classe concreta instanciar.

Quando usar: quando o código não deve depender de classes concretas específicas, mas sim de uma abstração — permitindo trocar ou expandir os tipos criados sem alterar o código cliente.

Implementação em Dart

Em Dart, o construtor factory pode servir como Factory Method: o código cliente usa `Notificacao()`, e o construtor decide internamente qual tipo concreto retornar com base em um parâmetro.

```

abstract class Notificacao {
  void enviar(String mensagem);

  // Factory Method: decide qual subclasse criar
  factory Notificacao(String tipo) {
    if (tipo == 'email') return NotificacaoEmail();
    if (tipo == 'sms')   return NotificacaoSms();
    return NotificacaoPush();
  }
}

class NotificacaoEmail implements Notificacao {
  void enviar(String msg) => print('Email: $msg');
}

class NotificacaoSms implements Notificacao {
  void enviar(String msg) => print('SMS: $msg');
}

class NotificacaoPush implements Notificacao {
  void enviar(String msg) => print('Push: $msg');
}

```

```

}

void main() {
  var n = Notificacao('email');
  n.enviar('Bem-vindo!');    // Email: Bem-vindo!
}

```

Ponto-chave: o código cliente não precisa saber qual classe concreta está sendo criada — basta usar a interface `Notificacao`. Isso facilita adicionar novos tipos sem modificar o código existente.

4. Strategy (Comportamental)

Definição: define uma família de algoritmos, encapsula cada um deles e os torna intercambiáveis. Permite que o algoritmo varie independentemente dos clientes que o utilizam.

Quando usar: quando existem múltiplas variações de um comportamento e é preciso trocar o algoritmo em tempo de execução sem alterar a classe que o usa.

Implementação em Dart

O padrão usa uma interface (classe abstrata) para representar a estratégia, e classes concretas para cada variação. A classe que usa a estratégia recebe qualquer implementação e não precisa saber os detalhes.

```

abstract class DescontoStrategy {
  double calcular(double preco);
}

class SemDesconto implements DescontoStrategy {
  double calcular(double preco) => preco;
}

class DescontoVip implements DescontoStrategy {
  double calcular(double preco) => preco * 0.8; // 20% de desconto
}

class DescontoPromocional implements DescontoStrategy {
  double calcular(double preco) => preco * 0.5; // 50% de desconto
}

class Carrinho {
  DescontoStrategy estrategia;
  Carrinho(this.estrategia);

  double finalizar(double total) => estrategia.calcular(total);
}

void main() {
  var c = Carrinho(DescontoVip());
  print(c.finalizar(200.0)); // 160.0

  c.estrategia = DescontoPromocional();
}

```

```
    print(c.finalizar(200.0)); // 100.0
}
```

Ponto-chave: o Strategy é uma aplicação direta de polimorfismo — a classe Carrinho trabalha com DescontoStrategy sem conhecer a implementação concreta. Trocar a estratégia em runtime é simples e seguro.

5. Observer (Comportamental)

Definição: define uma dependência de um-para-muitos entre objetos, de modo que, quando um objeto muda de estado, todos os seus dependentes são notificados e atualizados automaticamente.

Quando usar: quando uma mudança em um objeto exige que outros objetos também sejam alterados, sem saber quantos ou quais são esses objetos — como sistemas de eventos, alertas e notificações.

Implementação em Dart

O Observer usa duas abstrações: o Observavel (também chamado de Subject), que mantém a lista de ouvintes, e o Observador (Observer/Listener), que define o método chamado ao receber uma notificação.

```
abstract class Observador {
    void atualizar(String evento);
}

class SistemaDeAlertas {
    List<Observador> _observadores = [];

    void registrar(Observador o) => _observadores.add(o);
    void remover(Observador o)  => _observadores.remove(o);

    void notificar(String evento) {
        for (var o in _observadores) o.atualizar(evento);
    }

    void detectarProblema(String problema) {
        print('[Sistema] Problema detectado: $problema');
        notificar(problema);
    }
}

class AlertaEmail implements Observador {
    void atualizar(String e) => print('Email enviado: $e');
}

class AlertaLog implements Observador {
    void atualizar(String e) => print('Log registrado: $e');
}

void main() {
    var sistema = SistemaDeAlertas();
    sistema.registrar(AlertaEmail());
}
```

```
sistema.registrar(AlertaLog());
sistema.detectarProblema('CPU acima de 90%');
// Email enviado: CPU acima de 90%
// Log registrado: CPU acima de 90%
}
```

Ponto-chave: o SistemaDeAlertas não conhece os observadores concretos — apenas a interface Observador. Novos tipos de alerta podem ser adicionados sem modificar a classe existente.

6. Padrão MVC (Model-View-Controller)

MVC é um padrão arquitetural que divide a aplicação em três camadas com responsabilidades bem definidas, facilitando a manutenção e o teste do código.

Model	Representa os dados e as regras de negócio. Não sabe nada sobre a interface.
View	Exibe os dados ao usuário. Não processa lógica de negócio.
Controller	Recebe as ações do usuário, processa com o Model e atualiza a View.

Em Dart, o MVC é implementado com classes separadas para cada camada. O Controller coordena as interações, o Model mantém o estado e a View (em projetos de console) realiza a impressão ou captura de entrada.

7. Padrões DAO e VO

VO — Value Object

Um Value Object (VO) é uma classe simples que representa um conjunto de dados sem lógica de negócio complexa. É usada para transportar dados entre camadas da aplicação. Em Dart, geralmente possui atributos privados com getters e um método toString() para exibição.

```
class ProdutoVO {
  final String nome;
  final double preco;
  ProdutoVO({required this.nome, required this.preco});
  String toString() => '$nome - R$ ${preco.toStringAsFixed(2)}';
}
```

DAO — Data Access Object

O DAO isola toda a lógica de acesso a dados (leitura, escrita, busca) em uma classe dedicada. O restante do sistema nunca acessa o banco de dados diretamente — sempre por meio do DAO. Isso facilita trocar a fonte de dados (arquivo, banco, API) sem impactar o restante do código.

```
class ProdutoDAO {  
    List<ProdutoVO> _produtos = [];  
  
    void salvar(ProdutoVO p) => _produtos.add(p);  
  
    ProdutoVO? buscarPorNome(String nome) {  
        try {  
            return _produtos.firstWhere((p) => p.nome == nome);  
        } catch (_) {  
            return null;  
        }  
    }  
  
    List<ProdutoVO> listarTodos() => List.unmodifiable(_produtos);  
}
```

8. Conclusão

Padrões de projeto são ferramentas conceituais que elevam a qualidade do design orientado a objetos. Cada padrão resolve um problema específico: o Singleton controla instâncias únicas, o Factory Method desacopla a criação de objetos, o Strategy torna algoritmos intercambiáveis, o Observer sincroniza objetos de forma desacoplada, o MVC separa responsabilidades em camadas arquiteturais, e o DAO/VO isola o acesso a dados.

Em Dart, recursos como classes abstratas, interfaces (implements), construtores factory e membros estáticos tornam a implementação desses padrões natural e direta. O domínio desses padrões é fundamental para escrever código orientado a objetos profissional, manutenível e extensível.